

HC HANDBOOK

#algorithms

Apply algorithmic thinking strategies to solve problems and effectively implement working code.

Please fill out this [Google Form](#) to submit suggestions for this page (i.e. typos, study guide contributions, useful applications, etc.). We are actively seeking feedback on anything big or small to make the HC Handbook as useful as possible.

Defining the HC

Pitch

Using recipes to solve problems.

Paragraph Description

An algorithm defines a set of steps that lead a given input to produce the desired output. The steps of a proper algorithm are well-ordered, clear, unambiguous, and effective (doable). Additionally, the algorithm should be robust enough to handle a range of inputs, rather than a small set of specific cases. It should also be as simple and efficient as possible, avoiding redundancies. Lastly, a proper algorithm should terminate in a finite number of steps. These properties can be achieved by systematically incorporating conditional steps (in which a decision must be made) and iteration (repeated steps), usually requiring careful testing and troubleshooting (“debugging”).

Familiarity with various algorithmic strategies, such as brute-force methods, greedy algorithms, and divide-and-conquer approaches, can certainly broaden one’s problem-solving repertoire. These algorithmic strategies can be used to efficiently solve many real-world problems,



from connecting individuals on social media, to controlling self-driving cars, to sequencing DNA. Algorithms are the most powerful when they can be interpreted and run by computers. Thus, writing and explaining computer code helps to boost one's algorithmic thinking and problem-solving capabilities.

Categorization

Foundational Concept | Formal Analyses | Thinking Creatively | Solving Problems

Cornerstone Introduction

Class: Formal Analyses | Semester One

Unit: Logical Thinking

Big Question: How is free choice manipulated?

General Example

You need to assign students to residence hall rooms. Roommates must have the same gender and should have some common interests but should not be from the same country and should not have identical interests (you want students to be exposed to new influences). You can manually assign students to rooms, but this is complicated and tedious. Worse, you would have to do it all again next year. To address this, you decided to develop an algorithm that can be automated by computer code. Firstly, the algorithm takes an input of relevant characteristics from students, including details about living preferences, habits, and interests. You decide to use drop-down menus for students to input their selections because a finite set of pre-defined inputs makes your algorithm more feasible to write compared to free-form student inputs. Based on the data and parameters, the algorithm generates a potential list of matches for each student and assigns rooms starting from the students with the least options, updating the number of available matches as rooms are being assigned. Ultimately, the algorithm generates a list of room assignments that satisfy the constraints; if it is impossible to do so, the algorithm returns a message to notify you. You implement the algorithm using a computer program, which assigns roommates effectively and efficiently and can be used for each year's room assignment given similarly-formatted data on student characteristics. The code is complete with thorough

documentation and instructions for running, allowing your colleagues to understand the process.

Footnote: *Having identified a problem that benefits from an algorithmic solution, an appropriate process is developed and explained to assign students to rooms. The details of the algorithm are identified, explained, and justified: input, internal logic, output, and error message*

Is it this HC?

Is #algorithms the right HC? If the answer to the following questions is yes, #algorithms could be useful:

1. Does the work present a clear, step-by-step process or procedure to solve a problem, including the inputs and outputs?
2. Does the work evaluate one or more algorithmic strategies to approach a problem?

However, depending on the focus of the work, there might be other applicable HCs to consider. Consult the following table listing possible related HCs.

The following HC might apply if the work:

- | | |
|----------------------|--|
| #breakitdown | <ul style="list-style-type: none"> • breaks down a big problem into subproblems; in the context of algorithms, can include creating functions that compartmentalize the code and analyzing how the subproblems might “come together” in the program |
| #heuristics | <ul style="list-style-type: none"> • uses a ‘short-cut’ or a ‘rule-of-thumb’ strategy to solve the problem. • employs heuristics within the algorithm’s steps. For example, greedy algorithms typically use a heuristic to decide on the “best” action to take at each step (locally). |
| #optimization | <ul style="list-style-type: none"> • uses an appropriate optimization process or technique to obtain an locally or globally optimal solution to a problem, which may be |

through algorithmic strategies such as greedy or brute force.

#modeling

- uses a simulation (which could be an algorithmic process) to derive the implications or predictions of a model and pays particular focus to the assumptions of the model and the usefulness of the results. .

Self Study

Try answering the questions on your own before checking the example answers.

What are the requirements for a good algorithm?

Answer: The algorithm needs to be well-ordered, clear, unambiguous, effective, finite, and generalizable.

What does it mean for an algorithm to be clear and unambiguous? Why is this important?

Answer: The steps must be clear both for the computer to understand what and how to execute the operations AND for the user/coder. Computers are often 'dumb' as they lack intuition. We need to provide them with specific details so that they are able to perform the task accurately. This is important because otherwise the algorithm may fail to produce a correct result. For the user/coder, clear and unambiguous code eases debugging and reproducing the code when read by someone else.

What does it mean for an algorithm to be finite? Why is this important?

2

Answer: The algorithm contains a limited number of steps and each of them takes a finite amount of time to be executed. This way we guarantee that the algorithm always produces an output. One way to do this is to have a termination condition. For example, when working with ‘while’ loops we should avoid infinite loops. In some cases, it may be acceptable for an algorithm to run continually as long as it produces some output in finite time as it keeps running. An example of this could be a real-time monitoring system, where an output of a monitoring report is produced within finite time intervals.

👉 **What does it mean for an algorithm to be generalizable? Why is this important?**

Answer: An algorithm should be general enough so that it ‘works for all problems of a specific form’ as well as for a variety of input sets – this property can also be referred to as the robustness of the algorithm. The algorithm should also produce a correct result even when the set of inputs presents an edge case. The property of generalizability is related to another important property of ‘doability of operations’ since the algorithm must work for any appropriate type of input.

👉 **What does it mean for an algorithm to have effectively computable operations? Why is this important?**

Answer: Each step must be possible to perform otherwise the algorithm will be ineffective as it will get stuck and be unable to use the inputs to produce an output. This is also related to generalizability: the algorithm should be able to take in a wide range of inputs to transform without getting stuck.

👉 **What are some different ways to measure an**

algorithm's efficiency? Why is this important?

Answer: There are many different ways to measure an algorithm's efficiency. The most common approaches are considering the (1) number of steps, (2) runtime, and (3) memory/space usage. However, we are not interested just in the number of steps, runtime, or memory usage. We are interested in how the number of steps **scales** with the input size. Perhaps it may be a linear relationship between them: if the input is an array of n integers, and we simply have a single loop through the list, then the number of steps is also n . If we have a nested for-loop that goes through the list twice, then the number of steps is n^2 ; the number of steps scales in a quadratic behavior as input size increases. This is important because if we are testing an algorithm on a small input size and it runs very quickly, we need to know what would happen if we increase the input.

👉 **What are the following types of algorithms and are they guaranteed to always find the optimal solution? Give examples of problems that each algorithm type can solve: Brute-force, greedy algorithms, divide-and-conquer.**

Answer:

- **Brute force algorithm:** A brute force algorithm involves a systematic approach to try every possible solution to a problem until the best one is found. Brute force algorithms are not guaranteed to always find the globally optimal solution, particularly for problems with large input sizes because the computer might not have the capacity to get to the correct solution in a specific amount of time. One example is finding the shortest path between two points in a graph. A brute force approach would involve trying every possible path between the two points. Note that this can quickly become infeasible for large graphs. In such cases, a more efficient algorithm such as Dijkstra's algorithm or A* search algorithm may be used instead.

- **Greedy algorithm:** A greedy algorithm is one that makes locally optimal choices at each step in the hope of finding a globally optimal solution. These algorithms are often used for optimization problems and can be more efficient than brute force algorithms. However, they are not always guaranteed to find the optimal solution, and may get stuck in local optima. An example of a problem that can be solved using a greedy algorithm is the knapsack problem, where a set of items with different values and weights must be selected to maximize the total value while staying within a weight limit.
- **Divide-and-conquer algorithm:** A divide-and-conquer algorithm is one that breaks a problem down into smaller subproblems that can be solved independently. These subproblems are then combined to form a solution to the original problem. An example of a problem that can be solved using a divide-and-conquer algorithm is sorting a list of numbers using merge sort.

👉 Does an application of #algorithms require code?

Answer: Certainly not! Any time you are using a systematic process (with inputs, outputs, and steps) to solve a problem, you are using algorithmic thinking. Good algorithms can be written in simple natural language or pseudo code. They can also be visualized in a flowchart. Regardless of the format, a strong algorithm should meet the same features: well-ordered, clear, unambiguous, effective, finite, and generalizable.

👉 What does it mean to test an algorithm? What are different ways you can test an algorithm?

Answer: Testing an algorithm is essential to make sure that it works effectively as desired (that it produces the desired outputs for all possible inputs). It's usually best to begin tests with very simple cases for which you know what the expected output should

be: run the algorithm on an easy/simple input to ensure that the output matches expectations. The algorithm designer must then consider possible edge cases, failure modes, or ways to break the algorithm and continue more thoughtful tests to ensure the algorithm is robust enough to handle a wide variety of inputs. Here are a few common testing approaches:

Unit testing: This involves testing individual components or functions of the algorithm in isolation, using a set of inputs and expected outputs. Unit tests can be automated and run frequently during the development process.

Integration testing: This involves testing how different components or functions of the algorithm work together. Integration tests can uncover issues that may arise when different parts of the algorithm are combined.

Regression testing: This involves re-testing the algorithm after changes or updates have been made, to ensure that existing functionality has not been affected.

Acceptance testing: This involves testing the algorithm against a set of requirements or specifications provided by the client or end user. Acceptance tests are designed to ensure that the algorithm meets the desired functionality and performance criteria. Acceptance testing may be conducted in line with test-driven development, where tests are written before the code is written. This helps ensure that the code is working correctly and that it meets the desired functionality.

Upon testing the algorithms and identifying errors, you can engage in **debugging**, the process of finding and fixing errors or bugs in your code. You can use print statements and debuggers to step through your code and see what is happening at each step. This can help you identify and fix errors in your code.

👉 **What are different ways you can describe an algorithm's runtime?**

2

Answer: Complexity: Mathematical notation of Big-O, Big-Ω, Big-Θ are used to describe the algorithm's time complexity as a function of the input size. Big-O describes the asymptotic upper bound, or the worst-case scenario time complexity. Big-Ω describes the asymptotic lower bound or the best-case scenario. Big-Θ describes both the upper and lower bounds or tight bounds of the time complexity. It provides a range within which the growth rate of the algorithm's runtime falls.

Empirical analysis: This involves measuring the actual runtime of the algorithm for different input sizes and comparing it to the expected runtime based on the algorithm's time complexity analysis.

Benchmarking: This involves comparing the runtime of the algorithm to other algorithms solving the same problem, under similar conditions.

👉 **List at least three strategies you can use to understand how a complicated algorithm works.**

Answer:

1. Break the algorithm down into smaller components. Errors may be more easily identified when observing tractable parts.
2. If the algorithm is implemented in code, add “print” statements along the way to see what values certain variables take at different stages in the algorithm.
3. Trace the algorithm. Tracing an algorithm means walking through it step-by-step, manually calculating the output for a given input. This can help you understand how the algorithm processes information, how it makes decisions, and how it arrives at the final output. You can also experiment with the algorithm using different inputs.
4. Identify the algorithm's inputs and outputs. Then, test the algorithm against edge cases or unusual inputs.
5. Find out more about the algorithm by reading the algorithm's documentation, looking for examples of the algorithm in action, and discussing the algorithm with others.

6. Create visual representations of the algorithm. Visualizations such as flow charts or diagrams can help you understand how the different parts of the algorithm are interconnected. This can help you identify patterns, identify inputs and outputs, and see how the algorithm processes information.

👉 **List at least 2-3 strategies to debug code that isn't working properly.**

Answer:

1. Add print statements at different points in your code to see what values variables hold, which functions are being called, and where errors might be occurring.
2. Use a debugger to step through your code line by line, inspect variables, and identify where errors are occurring.
3. Read error messages carefully to understand what went wrong.
4. Write pseudocode before writing actual code and test each part of the code as you write it.

💡 Applying the HC

Guided Reflection

- Have you identified the input, output, and set of steps of the algorithm?
- Have you ensured that the steps adhere to desired properties of good algorithms, such as clarity, finiteness, robustness, termination, and efficiency, and included justification?
- Have you identified what type of algorithm is being used?
- Have you justified your choice of algorithm or type of algorithmic strategy, including the way that steps are ordered and broken down (possibly using functions)?
- Have you ensured that the algorithm fulfills its intended functionalities by demonstrating test cases, including possible edge cases?

- Have you included a sufficient amount of comments (using in-line coding comments and/or holistic explanations) to demonstrate that you have a deep grasp of the algorithm?
- Have you discussed the algorithm's complexity or efficiency?
- Have you proposed or implemented changes to improve an algorithm's efficiency? (e.g., utilizing iteration to avoid redundancy)
- Have you created a flowchart to visualize the algorithm, adhering to proper flowchart conventions?
- Have you justified the choice of data structures utilized in the algorithm?
- Have you adhered to conventions for readable algorithms, such as using appropriate variable names and comment styles?

Common Pitfalls

- The application misidentifies or fails to identify the outputs and inputs.
- The steps of the algorithm are not specified in sufficient detail.
- The steps of the algorithm are vague, ambiguous, ineffective, or incomplete.
- The algorithm does not work as intended.
- The application does not include sufficient tests or does not account for edge cases when testing.
- The algorithm does not include sufficient comments to explain how it works.
- The efficiency of the algorithm has not been sufficiently considered.
- The algorithm uses inappropriate data structures.
- The application involves inappropriate or non-descriptive variable names.

Applying the HC at Minerva

- This HC is applicable to most CS core and concentration classes.
 - Across machine-learning concentrations, different algorithmic strategies are involved, such as supervised vs unsupervised.
 - Algorithms may be used in line with **#modeling** to create simulations for a wide variety of real-world system, including traffic, flooding, and biological systems.

- The HC may be applicable in business contexts, as organizations and companies need to construct and implement algorithms to solve problems or perform tasks that occur on a regular basis. For example, consider a company's customer service department. An algorithm for employees to follow would be helpful to ensure that they are dealing with questions from customers in a systematic and consistent manner. The algorithm's input would be the customer's questions or complaints, and the desired output would be some sort of resolution. Constructing such an algorithm would require deeply understanding the huge variety of customer inquiries. The algorithm would need decision nodes (like if-statements) that lead down different branches depending on the complaint. More thought would need to be given to various edge cases that fall outside the usual categories of inquiries and require a unique solution. The algorithm should have a guideline for addressing such unusual novel cases. Once the algorithm is developed to a sufficient level of detail and thoroughly tested out, even a computer can follow it! Does the company even need a human to handle customer service anymore?

Applying the HC Outside of Minerva

- Algorithmic thinking is crucial in several aspects of daily life and across many different fields. Any time you are using a set of instructions to tackle a difficult problem, you are likely employing the principles of algorithmic thinking. Here are a few examples:
 - It can help you develop and follow recipes for cooking and baking.
 - It is used in industries like transportation, logistics, and supply chain management. In these fields, algorithms are used to optimize delivery routes, cut costs, and boost productivity.
 - Algorithmic trading is an approach in finance to evaluate the stock market and create transactions.
- In healthcare, algorithmic thinking is used to build clinical decision support systems that help doctors make diagnoses and plan treatments. For diseases that have a highly complex diagnosis, algorithms are especially valuable to establish protocols. **This article** in the National Library of Medicine

provides one example: *A clinical approach to diagnosis of autoimmune encephalitis*. In this article, expertise from a large team of professionals has been used to construct an algorithm to provide autoimmune encephalitis diagnosis guidelines for medical practitioners. **This flow chart** contains a visualization of the algorithm, complete with several decision nodes (“if-statements”) that serve to handle the wide variety of factors that need to be considered in the process, and clear outputs for each case.

- An application of algorithms in music! Common algorithmic strategies, like brute force, are widely applicable outside of computer science. Musicians can even adopt algorithmic composition techniques! A famous example is in John Williams' score for the 1977 Steven Spielberg film, 'Close Encounters of the Third Kind.' The five tone sequence used throughout the movie was found through a brute force approach by listing all 130k+ permutations of 5 notes on the western musical scale. He didn't actually listen to them all, but rather to a subset of about 400, then selected the one he liked. Those interested in this can look up old interviews about. Here is one clip from **the most famous scene**.
- Check out the HC Handbook for **#optimization** for more real-world practical applications of algorithms in optimization settings, including picking what to eat, navigating the grocery store, and planning your schedule.

Key Resources

- Malan, D. J. (2013, May 20). [TED-Ed]. What's an algorithm? [Video]. YouTube. <https://www.youtube.com/watch?v=6hfOvs8pY1k>
 - A simple introduction to the concept of algorithms and how to use them to solve problems.
- DataCamp Team. (2023, Sept). What is an Algorithm? DataCamp. <https://www.datacamp.com/blog/what-is-an-algorithm>
 - A quick but comprehensive introduction to algorithms, providing a definition, examples, use-cases, and features of

good algorithms.

- Walia, R. K. (n.d.). Algorithm & Flowchart Manual for Students. https://course-resources.minerva.edu/uploaded_files/mu/00306798-0780/algorithm-and-flowchart-manual-algorithm.pdf
 - This resource describes why flowcharts are helpful and supplies some best practices and conventions to follow.
- Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2022, Dec). Dive into Deep Learning. https://d2l.ai/chapter_introduction/index.html#
 - The first chapter of this text provides an exciting and accessible introduction to machine learning.
- Shafkat, I. & Terrana, A. (2022, Nov 26). Optimization Cornerstone Guide. Minerva University. https://course-resources.minerva.edu/uploaded_files/mu/00298160-6576/optimization-cornerstone-
 - The guide gives an in-depth overview of optimization and discusses optimization in algorithmic contexts.